Security Audit Report

Diffuse Prime
Code commit: `e4bf94812ff041e63dd92262b17e767ec5c286c4`

Mikhail Voronov Valery Antopol

October 14, 2025

1 Introduction

This report documents an internal cross-audit performed on the Diffuse Prime protocol. The audit assessed the state of the codebase at commit `e4bf94812ff041e63dd92262b17e767ec5c286c4`. The objective of this internal review was to identify potential vulnerabilities, verify correctness, and provide recommendations for improvement.

2 Scope

All code of the protocol is presented in the scope of the audit.

3 Summary of Findings

Provide a short summary:

- Total issues found: 17

Critical: 0
High: 5
Medium: 6
Low: 4
Info: 2

4 Detailed Findings

4.1 High

4.1.1 `leftPointer` management ignores finished strategies in deposit

Severity: High

Description:

This loop updates debt only from `leftPointer` to end, ignoring finished strategies (left of pointer). This allows a new depositor to get historical `accPointsPerShare` from finished strategies without paying for it. Their debt for finished strategies remains 0, so on withdrawYield they claim `newShares * accPointsPerShare - 0`, stealing yield earned by earlier lenders.

Attack: deposit after strategy ends but before yield withdrawn.

Let's assume the following:

- Strategy #0: finished, `accPointsPerShare = 1000` (accumulated yield from old lenders)
- Strategy #1: active
- `leftPointer = 1` (moved past #0)

Then one deposits 100 after strategy #0 ends:

- Loop runs only from `i=1` to end
- For strategy #0: debt NOT updated (loop skips it)

Result: `userShares = 100, debt[0] = 0, accPointsPerShare[0] = 1000`.

On `withdrawYield(strategyId=0): pending = 100 * 1000 / PRECISION - 0`. Attacker steals yield earned by earlier lenders.

Recommendation:

–

Location:

src/libs/BaseLogicLib.sol#L115

Resoution: Fixed. Left pointer renamed to first active strategy. This pointer is now moved only when all yield is withdrawn from a strategy, and new yield isn't generated in such inactive strategies.

4.1.2 `leftPointer` management ignores finished strategies in withdraw

Severity: High

Description:

This loop transfers pending to `unpaidYield` only from `leftPointer` to end, ignoring finished strategies. On full `withdraw (amount == userShares)`, pending for finished strategies is NOT moved to `unpaidYield`. After shares become 0, that yield is lost forever since `pending = 0 * accPointsPerShare - debt = 0`.

User cannot recover it via `withdrawYield` because pending calculation requires non-zero shares.

Let's assume the following:

- Strategy #0: finished, `accPointsPerShare = 1000`
- `leftPointer = 1` (moved past #0)
- User: `userShares = 100, debt[0] = 50`

- Pending for strategy #0: `100 * 1000 / PRECISION - 50 = X` (some yield)

User calls `withdraw(100)` (full withdraw of shares):

- Loop runs only from `i=1` to end (skips #0)
- For strategy #0: pending NOT moved to unpaidYield
- userShares become 0, debt[0] remains 50

Result: pending `X` for strategy #0 is lost forever. Later `withdrawYield(0)`: `pending = 0 * 1000 - 50 = 0`.

Recommendation:

–

Location:

src/libs/BaseLogicLib.sol#L178

Resoution:

Fixed, same as 4.1.1. Finished strategies are still ignored in withdrawal/deposit, but no yield can be withdrawn from them.

4.1.3 Possible bug in lender `withdrawYield` algorithm

Severity: High

Description:

Code here computes total payout as unpaid + pending, then clamps it to the vault balance. However, it always deducts the entire ledgered unpaid `decreaseUnpaidYield(..., unpaidYield)` and resets pending by setting `debt = shares * accPointsPerShare`, even when the actual payout is smaller than total accrued. This burns the unpaid remainder under liquidity shortages, making payouts timing-dependent and unfair.

Let's assume the following:

- `unpaidYield = 100` (ledger)
- `pending = 50` (accumulated)
- `total = 150`
- `remainedBalance = 80`

The following happens:

1. `yield = min(150, 80) = 80`
2. `decreaseUnpaidYield(..., 100) -> ledger` becomes 0
3. `debt = shares * accPointsPerShare -> pending` becomes 0
4. User receives 80

The result:

- Paid: 80
- Lost: 70 (20 from ledger + 50 from pending)

Recommendation:
–
Location:
src/libs/BaseLogicLib.sol#L302
Resoution:
Fixed. There can now be zero active strategies, as it should be.

4.1.4 Missing blockhash verification in borrower position activation

Severity: High
Description:
Only hash comparison is performed, but actual blockhash validation is missing (unlike in the liquidate function). Without on-chain blockhash verification, fake block data could be used if SGX/oracle is compromised.
Recommendation:
Should add loop to validate: `blockNumber <= currentBlock - 256, getBlockHash(blockNumber) == providedHash`.
Location:
src/libs/BorrowLogicLib.sol#L298
Resoution:
Fixed.

4.1.5 Incorrect rounding-up in fees calculation

Severity: High
Description:
Adding +1 systematically creates `baseAssetDeficit` on every position close, even with perfect economics. For N positions, deficit accumulates to N wei. With millions of micro-positions, this becomes significant DoS vector (blocks lender withdrawals).
Recommendation:
Should use proper `Math.mulDiv` with consistent rounding instead.
Location:
src/libs/BorrowLogicLib.sol#L382
Resoution:
Fixed. `Math.mulDiv` is used for all calculations.

4.2 Medium

4.2.1 "pointer to first active" semantics broken

Severity: Medium
Description:

When all strategies are finished, this sets `leftPointer = strategyLength - 1` instead of `strategyLength`. This breaks the "pointer to first active" semantics and causes deposit/withdraw loops to process the last (already finished) strategy while still skipping all earlier finished strategies, masking problems of finished strategies.

Recommendation:

Suggest it to be: `leftPointer = strategyLength` (empty active range).

Location:

src/libs/BaseLogicLib.sol#L50

Resoution:

Fixed along with 4.1.1 and 4.1.2.

4.2.2 Incorrect bytes reading with `0x20` offset

Severity: Medium

Description:

Reading bytes without `+0x20` offset reads from header instead of data, causing validation to compare wrong memory slots. All three fields `(hash1, hash2, mrEnclave)` are read from incorrect offsets.

Recommendation:

Should be: `mload(add(add(output, 0x20), OFFSET))` to skip the length prefix.

Location:

src/libs/BorrowLogicLib.sol#L422

Resoution:

Incorrect finding, `0x20` addition is already taken into account

4.2.3 Overflow risk in `updateLenderPoints`

Severity: Medium

Description:

Multiplication of several terms risks overflow.

Recommendation:

Consider using `Math.mulDiv` for safe intermediate calculations.

Location:

src/libs/BaseLogicLib.sol#L92

Resoution:

Fixed.

4.2.4 Overflow risk in `getAccruedLenderYieldOneStrategy`

Severity: Medium

Description:

Same overflow risk as in `updateLenderPoints`.

Recommendation:

Consider using `Math.mulDiv`.

Location:

src/libs/BaseLogicLib.sol#L258

Resoution:
Fixed.

4.2.5 Possible swaps through untrusted adapters

Severity: Medium
Description:
Borrow activation performs swaps along a curator-defined forward route using adapters that may be untrusted. The borrower chooses minStrategyToReceive, on activation we only check actual received $\geq$ this borrower-provided minimum. If the minimum is set artificially low or the route yields a poor price, activation can pass while leaving lenders undercollateralized. Even immediate liquidation via the reverse route may fail to recover principal if the reverse route is unfavorable or reverts.
Recommendation:
The oracle should always off-chain call previewBorrow and enforce independent price/slippage limits before activation. Additionally, whitelist/audit adapters and enforce timelocks on route changes to reduce curator or adapter risk.
You can add this check:

- `minStrategyReceived >= oracleMinStrategyToReceive`
- `strategyBalanceReceived >= oracleMinStrategyToReceive`

Location:
src/libs/BorrowLogicLib.sol#L263
Resoution:

- Added timelock for route changes.
- Borrower sets slippage limits when creating a position.

4.2.6 Possible proof reuse

Severity: Medium
Description:
Payload hash only covers `liquidationPrice`, allowing proof reuse across different positions/users/strategies with same `liquidationPrice` within 256 blocks.
Recommendation:
Suggested to include: `vault, positionId, user, strategyId, collateralType, collateralAmount, assetsToBorrow, minStrategyToReceive, deadline`, and `blocksHash` to prevent replay attacks and protect against bugs in off-chain oracle logic.
Location:
src/libs/BorrowLogicLib.sol#L290
Resoution:
Fixed: added mapping for unique proofs, enough for v1.

4.3 Low

4.3.1 Inconsistent `vaultAndPositionId` type size

Severity: Low

Description:

Here, `vaultAndPositionId` is truncated to `uint32`, but in many other places it's used as `uint256`.

Recommendation:

Change `vaultAndPositionId` for consistency.

Location:

src/libs/BorrowLogicLib.sol#L500

Resoution: Fixed. Also, `MAX_BORROWER_POSITION_ID` added to check that the position id is within `uint32` borders.

Note: It is used as `uint256` for better architecture, storage and simplicity.

4.3.2 Confusing naming for a non-view function

Severity: Low

Description:

This function is not view and despite the name it does liquidation itself, and it could be called from `previewLiquidateQuote` which is not view as well and external.

Recommendation:

Looks like you should create an actual view-method for liquidatation preview.

Location:

src/libs/BorrowLogicLib.sol#L585

Resoution:

We can't do it a view method. It is common practice to do this way. Still, no one can execute it, since `previewLiquidateQuote` always reverts and external `previewLiquidate` needs `msg.sender == 0`. Also, external non-view methods are usually named similar to the view ones.

4.3.3 Allowance not revoked

Severity: Low

Description:

Allowance not revoked after buy and sell in Pendle Adapter.

Recommendation:

Consider adding: `TOKEN_IN.forceApprove(POOL, 0)` to revoke allowance. Same applies for all future adapters.

Location:

src/adapters/PendleAdapter.sol#L54 and src/adapters/PendleAdapter.sol#L65

Resoution:

Fixed.

4.3.4 Possible yield steal after maturity date

Severity: Low

Description:

Returning uint256.max allows unlimited deposits even after strategies finish. Combined with the bug when deposit ignores finished strategies' debt, an attacker can deposit large amounts after strategy completion and steal accumulated yield from earlier lenders.

Recommendation:

Consider limiting to `availableLiquidity` or 0 when there's unpaid yield on finished strategies.

Location:

src/facets/ERC4626Facet.sol#L238

Resoution:

Fixed along with 4.1.1 and 4.1.2.

4.4 Info

4.4.1 Poor Constants Naming

Severity: Info

Description:

Directly usage of constant values makes it difficult to trace the origin and the logic.

Recommendation:

It's better to name constants, otherwise it's difficult to understand this logic.

Location:

script/AddPendleStrategy.s.sol#L99

Resoution:

Fixed.

4.4.2 More logical place for setting liquidation price of a new position

Severity: Info

Description:

It seems `position.liquidationPrice = actualLiquidationPrice` should be here because it's currently only used in the emitted event and some offchain logic or frontend could potentially use it.

Recommendation:

Move setting `position.liquidationPrice` to this method.

Location:

src/libs/BorrowStorageLib.sol#L169

Resoution:

Valid, Fixed.

5 Disclaimer

This audit does not guarantee the absence of vulnerabilities. It represents a best-effort review based on the provided scope and timeframe.